

Keras 人工神经网络

Fashion MNIST 服饰分类与过拟合诊断

涵盖内容：

- ✓ Fashion MNIST 数据加载与归一化
- ✓ 多层感知机结构设计与参数量计算
- ✓ 多类分类输出层与损失函数选择
- ✓ ModelCheckpoint 与 EarlyStopping
- ✓ 学习曲线、混淆矩阵与错误样本分析

环境：VS Code + Python + TensorFlow/Keras

请先完成理论分析，再补全代码并开展对比实验

第 1 题 Fashion MNIST 服饰分类与过拟合诊断

考点：Sequential API • Dense • ReLU • Softmax • 交叉熵 • 回调函数 • 泛化能力

1.1 问题背景

某服装电商平台希望构建一个自动分类系统，将商品灰度图像划分为以下 10 个类别：

标签	类别	标签	类别
0	T-shirt/top	5	Sandal
1	Trouser	6	Shirt
2	Pullover	7	Sneaker
3	Dress	8	Bag
4	Coat	9	Ankle boot

现使用 Fashion MNIST 数据集完成该任务。数据集包含 70 000 幅 28×28 的灰度图像，其中 60 000 幅属于原始训练集，10 000 幅属于测试集。

你需要构建一个多层感知机，并通过训练曲线和错误样本分析判断模型是否出现欠拟合或过拟合。为了保证测试结果可信，测试集只能在模型设计和超参数选择完成后使用一次。

1.2 第一部分：数据准备与任务分析

引导性问题

1. 原始图像的像素值位于 0 到 255。为什么通常要将其缩放到 0 到 1？特征尺度会如何影响梯度下降？
2. 为什么应当从原始训练集中划分验证集，而不能直接使用测试集选择隐藏层数量、学习率和训练轮数？
3. 单个输入样本的形状为 (28, 28)。经过 Flatten 层后，其形状是多少？
4. 本任务中每幅图像只能属于一个类别，因此它属于二元分类、多标签分类还是多类分类？
5. 若标签采用 0 到 9 的整数形式，应使用 categorical_crossentropy 还是 sparse_categorical_crossentropy？说明理由。

代码任务

补全以下代码，加载数据、划分验证集并完成像素归一化：

```
1 import numpy as np
2 import tensorflow as tf
3
```

```
4 tf.keras.utils.set_random_seed(42)
5
6 # 加载 Fashion MNIST
7 fashion_mnist = tf.keras.datasets.fashion_mnist.load_data()
8 (X_train_full, y_train_full), (X_test, y_test) = fashion_mnist
9
10 # TODO: 将原训练集最后 5000 幅图像作为验证集
11 X_train = ...
12 y_train = ...
13 X_valid = ...
14 y_valid = ...
15
16 # TODO: 转换为 float32, 并将像素值缩放到 [0, 1]
17 X_train = ...
18 X_valid = ...
19 X_test = ...
20
21 print("训练集: ", X_train.shape, y_train.shape)
22 print("验证集: ", X_valid.shape, y_valid.shape)
23 print("测试集: ", X_test.shape, y_test.shape)
24 print("像素范围: ", X_train.min(), X_train.max())
```

验证点

程序输出应满足:

- 训练集包含 55 000 幅图像;
- 验证集包含 5 000 幅图像;
- 测试集包含 10 000 幅图像;
- 归一化后像素值位于 $[0, 1]$ 。

1.3 第二部分: 设计 MLP 并计算参数量

平台计划使用如下网络:

$28 \times 28 \rightarrow \text{Flatten} \rightarrow \text{Dense}(256, \text{ReLU}) \rightarrow \text{Dense}(128, \text{ReLU}) \rightarrow \text{Dense}(10, \text{softmax})$.

引导性问题

1. 第一个隐藏层的权重矩阵和偏置向量分别是什么形状？
2. 第二个隐藏层的权重矩阵和偏置向量分别是什么形状？
3. 输出层为什么需要 10 个神经元？为什么使用 softmax 而不是 10 个彼此独立的 sigmoid？
4. 分别计算三个 Dense 层的可训练参数量，并给出模型的总参数量。计算公式为

$$n_{\text{params}} = n_{\text{input}}n_{\text{output}} + n_{\text{output}}.$$

5. 如果删除两个隐藏层的 ReLU 激活函数，无论继续堆叠多少个 Dense 层，模型的整体表达能力会发生什么变化？

代码任务

```
1 model = tf.keras.Sequential([
2     tf.keras.layers.Input(shape=(28, 28)),
3     # TODO: 将二维图像展开为一维向量
4     ...,
5
6     # TODO: 添加包含 256 个神经元的 ReLU 隐藏层
7     ...,
8
9     # TODO: 添加包含 128 个神经元的 ReLU 隐藏层
10    ...,
11
12    # TODO: 添加适用于 10 类互斥分类的输出层
13    ...,
14 ])
15
16 model.summary()
```

将 `model.summary()` 显示的参数量与手工计算结果进行比较。如果结果不同，应检查是否遗漏偏置项。

1.4 第三部分：编译模型并使用回调函数训练

引导性问题

1. 为什么分类任务通常使用交叉熵，而不是均方误差？

2. Adam 优化器的学习率过大时，训练损失可能出现什么现象？学习率过小时又会出现什么现象？
3. EarlyStopping 中的 patience=5 表示什么？
4. 为什么早停应监控 val_loss，而不是只监控 loss？
5. restore_best_weights=True 与仅仅提前停止训练有什么区别？
6. ModelCheckpoint 和 EarlyStopping 是否可以同时使用？二者分别解决什么问题？

代码任务

补全模型编译和训练代码。最多训练 50 个轮次，但在验证损失停止改善时提前终止：

```
1 optimizer = tf.keras.optimizers.Adam(  
2     learning_rate=1e-3  
3 )  
4  
5 model.compile(  
6     # TODO: 选择适合整数类别标签的损失函数  
7     loss=...,  
8     optimizer=optimizer,  
9     metrics=["accuracy"],  
10 )  
11  
12 checkpoint_cb = tf.keras.callbacks.ModelCheckpoint(  
13     "best_fashion_mnist.keras",  
14     monitor="val_loss",  
15     save_best_only=True,  
16     mode="min",  
17 )  
18  
19 early_stopping_cb = tf.keras.callbacks.EarlyStopping(  
20     monitor="val_loss",  
21     patience=5,  
22     # TODO: 训练结束时恢复最佳轮次的参数  
23     restore_best_weights=...,  
24 )  
25  
26 history = model.fit(  
27     X_train,  
28     y_train,
```

```
29     epochs=50,  
30     batch_size=64,  
31     validation_data=(X_valid, y_valid),  
32     callbacks=[  
33         checkpoint_cb,  
34         early_stopping_cb,  
35     ],  
36 )
```

训练完成后回答：

1. 模型实际训练了多少个轮次？
2. 验证损失最低出现在哪个轮次？
3. 该轮次的训练准确率和验证准确率分别是多少？
4. 最后一个轮次是否就是验证性能最好的轮次？为什么？

可以使用以下代码寻找最佳轮次：

```
1 best_epoch = np.argmax(history.history["val_loss"]) + 1  
2 best_val_loss = np.min(history.history["val_loss"])  
3  
4 print("最佳轮次：", best_epoch)  
5 print("最低验证损失：", best_val_loss)
```

1.5 第四部分：绘制学习曲线并诊断模型

绘制训练集与验证集的损失曲线和准确率曲线：

```
1 import matplotlib.pyplot as plt  
2  
3 epochs = range(1, len(history.history["loss"]) + 1)  
4  
5 fig, axes = plt.subplots(1, 2, figsize=(12, 4))  
6  
7 # TODO: 在左图绘制训练损失和验证损失  
8 axes[0].plot(epochs, ..., label="train_loss")  
9 axes[0].plot(epochs, ..., label="val_loss")  
10 axes[0].set_xlabel("Epoch")  
11 axes[0].set_ylabel("Loss")  
12 axes[0].set_title("Loss curves")
```

```
13 axes[0].grid(True)
14 axes[0].legend()
15
16 # TODO: 在右图绘制训练准确率和验证准确率
17 axes[1].plot(epochs, ..., label="train_accuracy")
18 axes[1].plot(epochs, ..., label="val_accuracy")
19 axes[1].set_xlabel("Epoch")
20 axes[1].set_ylabel("Accuracy")
21 axes[1].set_title("Accuracy curves")
22 axes[1].grid(True)
23 axes[1].legend()
24
25 plt.tight_layout()
26 plt.show()
```

根据曲线判断模型属于以下哪一种情况，并给出证据：

1. 欠拟合：训练集和验证集性能都较差；
2. 拟合合理：训练集和验证集性能接近，验证损失稳定下降；
3. 过拟合：训练性能继续提高，但验证损失开始上升。

若出现过拟合，请从以下方法中选择两种，并解释其作用：

- 减少隐藏层神经元数量；
- 使用早停；
- 添加 L_2 正则化；
- 添加 Dropout 层；
- 收集或增强训练数据。

1.6 第五部分：最终评估与错误样本分析

模型设计完成后，加载验证性能最佳的模型，并且只在此时使用测试集：

```
1 best_model = tf.keras.models.load_model(
2     "best_fashion_mnist.keras"
3 )
4
5 test_loss, test_accuracy = best_model.evaluate(
6     X_test,
```

```
7     y_test,
8     verbose=0,
9 )
10
11 print("测试损失: ", test_loss)
12 print("测试准确率: ", test_accuracy)
13
14 # 预测类别概率
15 y_proba = best_model.predict(X_test, verbose=0)
16
17 # TODO: 沿类别轴取最大概率对应的类别
18 y_pred = ...
19
20 print("预测结果形状: ", y_pred.shape)
21 print("概率矩阵形状: ", y_proba.shape)
```

使用混淆矩阵分析模型容易混淆的类别:

```
1 from sklearn.metrics import (
2     confusion_matrix,
3     classification_report,
4 )
5
6 cm = confusion_matrix(y_test, y_pred)
7 print("混淆矩阵: ")
8 print(cm)
9
10 print("分类报告: ")
11 print(
12     classification_report(
13         y_test,
14         y_pred,
15         digits=4,
16     )
17 )
```

完成以下分析:

1. 找出混淆矩阵中数值最大的两个非对角元素。它们分别表示哪两个类别容易被混淆?
2. 比较各类别的 precision、recall 和 F1-score。哪个类别最难识别?

3. 对每个测试样本定义模型置信度：

$$c_i = \max_k \hat{p}_{ik}.$$

找出所有错误预测中置信度最高的 9 个样本。

4. 高置信度错误与低置信度错误相比，哪一种更值得重点检查？它们可能反映数据、标签或模型的哪些问题？

可使用以下代码显示错误样本：

```
1 class_names = [  
2     "T-shirt/top", "Trouser", "Pullover", "Dress",  
3     "Coat", "Sandal", "Shirt", "Sneaker",  
4     "Bag", "Ankle boot",  
5 ]  
6  
7 confidence = y_proba.max(axis=1)  
8 wrong_indices = np.where(y_pred != y_test)[0]  
9  
10 # TODO: 按置信度从高到低排列错误样本  
11 sorted_wrong = ...  
12 selected = sorted_wrong[:9]  
13  
14 fig, axes = plt.subplots(3, 3, figsize=(9, 9))  
15  
16 for ax, index in zip(axes.ravel(), selected):  
17     ax.imshow(X_test[index], cmap="gray")  
18     ax.set_title(  
19         f"True: {class_names[y_test[index]]}\n"  
20         f"Pred: {class_names[y_pred[index]]}, "  
21         f"p={confidence[index]:.2f}"  
22     )  
23     ax.axis("off")  
24  
25 plt.tight_layout()  
26 plt.show()
```

1.7 第六部分：对比实验

保持训练集、验证集和测试集划分不变，完成以下三组实验：

实验	隐藏层结构	学习率	早停
A	[64]	10^{-3}	使用
B	[256, 128]	10^{-3}	使用
C	[512, 256, 128]	10^{-3}	不使用

将实验结果填入下表：

实验	参数量	训练轮数	训练准确率	验证准确率	测试准确率
A					
B					
C					

根据实验结果回答：

1. 参数量最多的模型是否一定具有最高的测试准确率？
2. 哪个模型欠拟合的风险最大？依据是什么？
3. 哪个模型过拟合的风险最大？依据是什么？
4. 早停如何影响训练时间和泛化性能？
5. 如果只能部署一个模型，你会选择哪一个？请同时考虑准确率、参数量和过拟合风险，不能只比较训练准确率。

1.8 提交要求

提交内容应包括：

1. 完整且能够独立运行的 Python 代码；
2. 模型结构摘要及手工参数量计算过程；
3. 训练损失和准确率曲线；
4. 测试集准确率、混淆矩阵和分类报告；
5. 9 个高置信度错误样本的可视化结果；
6. 三组对比实验结果及不少于 300 字的分析结论。

1.9 评价标准

评价项目	分值
数据划分与归一化正确，无测试集泄漏	15
网络结构、输出层和损失函数选择正确	20
参数量计算正确	10
正确使用模型检查点和早停回调	15
学习曲线及过拟合诊断合理	15
测试评估与错误样本分析完整	15
对比实验结论有数据支持	10
总分	100

附：常见误区

1. 不要使用测试集选择训练轮数。验证集用于模型选择，测试集只用于最终泛化性能评估。
2. 整数标签不能直接搭配普通分类交叉熵。若标签是 0 到 9 的整数,应使用 `sparse_categorical_crossentropy`。
3. 不要遗漏像素归一化。未归一化的像素值会使优化过程更加困难，并可能要求重新调整学习率。
4. 不要把训练准确率作为唯一依据。训练准确率很高但验证损失上升，通常是过拟合信号。
5. 不要在每次对比实验中重新划分数据。所有实验必须使用相同的数据划分，否则结果不具备可比性。
6. 不要只报告总体准确率。混淆矩阵、分类报告和错误样本能够揭示总体准确率掩盖的类别差异。